

18. Le dessin écrit là où on le pose

Les huit rangées dans le « poser » lui-même — sans nom, et sans numéro.

1. Ce qu'il faut comprendre

Une case de l'écran, c'est **huit rangées de huit pixels**, jamais autre chose. La leçon d'avant leur donnait un nom — `Tuile BLOC = {...}` — puis posait ce nom. Mais un nom, à l'arrivée, **c'est un numéro** : `poser(2, 2, BLOC)` écrit 44 dans la carte de fond.

Ces huit rangées peuvent s'écrire **dans le `poser` lui-même**. Le compilateur grave la tuile, lui trouve son numéro, et l'écrit à la case demandée — il n'y a plus ni nom à inventer ni numéro à suivre, et le dessin est **là où il apparaît**.

Ce n'est pas un remplacement, c'est un choix. Ce qui sert PARTOUT mérite son nom : le sol ci-dessous est posé vingt fois, et `SOL` le dit à chaque ligne. Ce qui ne sert QU'À UN ENDROIT n'apprend rien de plus en s'appelant `TUILE3`, et oblige à lire deux endroits pour en comprendre un seul.

Le même dessin ne se grave qu'une fois. Écris deux fois le même nuage à dix lignes d'écart : le compilateur reconnaît qu'il l'a déjà, et ne dépense qu'une tuile. Il reconnaît aussi un dessin déjà déclaré en `Tuile`, **à travers les deux écritures** — `"21111112"` et `"+-+-----+"` sont les mêmes pixels. La console n'en tient que 256 : les gaspiller ne se verrait qu'au jour où il n'y en aurait plus.

Cela marche aux quatre endroits où un numéro de tuile est attendu : `poser`, `poserPanneau`, `sprite`, et `sprite16` — qui veut alors **seize rangées de seize signes**, et les découpe lui-même en quatre tuiles. Ailleurs, des accolades restent une faute, et le compilateur le dit avec sa ligne.

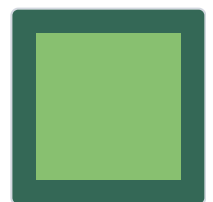
Une chose que le dessin sur place ne fait pas : **il n'apparaît pas dans l'atelier**. Celui-ci lit les `Tuile NOM = {...}` du texte, et il ne saurait pas où réécrire un dessin qui n'a pas de nom. Ce qu'on veut peindre à la souris, on le nomme.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1436 octets** de cartouche.

1. UN DESSIN — SOL

```
Tuile SOL = {
  "22222222",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "22222222",
};
```



à l'écran

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(4, 2, "SUR PLACE");

    // Le sol sert partout : il garde son nom.
    for (uint8_t x = 0; x < 20; x++) {
        poser(x, 12, SOL);
    }

    // La porte ne sert qu'ici : ses huit rangées sont ici.
    poser(9, 11, {
        "#####",
        "#-----#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#####",
    });

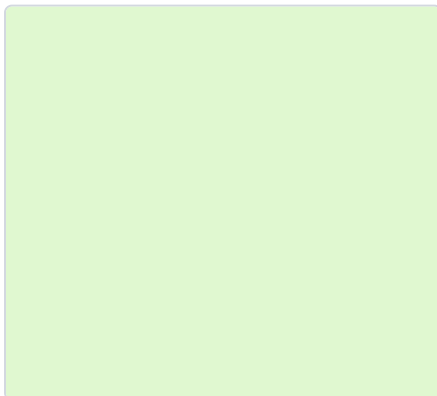
    // Le dessin de SOL, redit dans l'autre écriture : aucune tuile de plus.
    poser(3, 10, {
        "+++++++",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+++++++",
    });

    while (true) {
        image();
    }

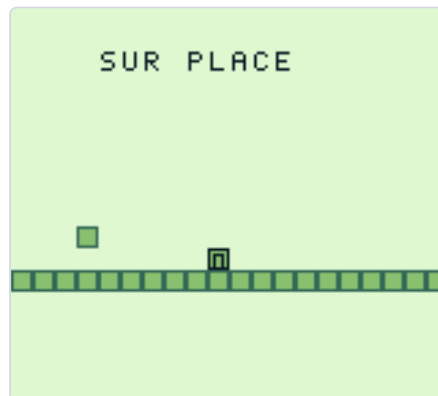
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un sol de vingt cases, une porte posée au-dessus, et à gauche un pavé qui reprend le dessin du sol.

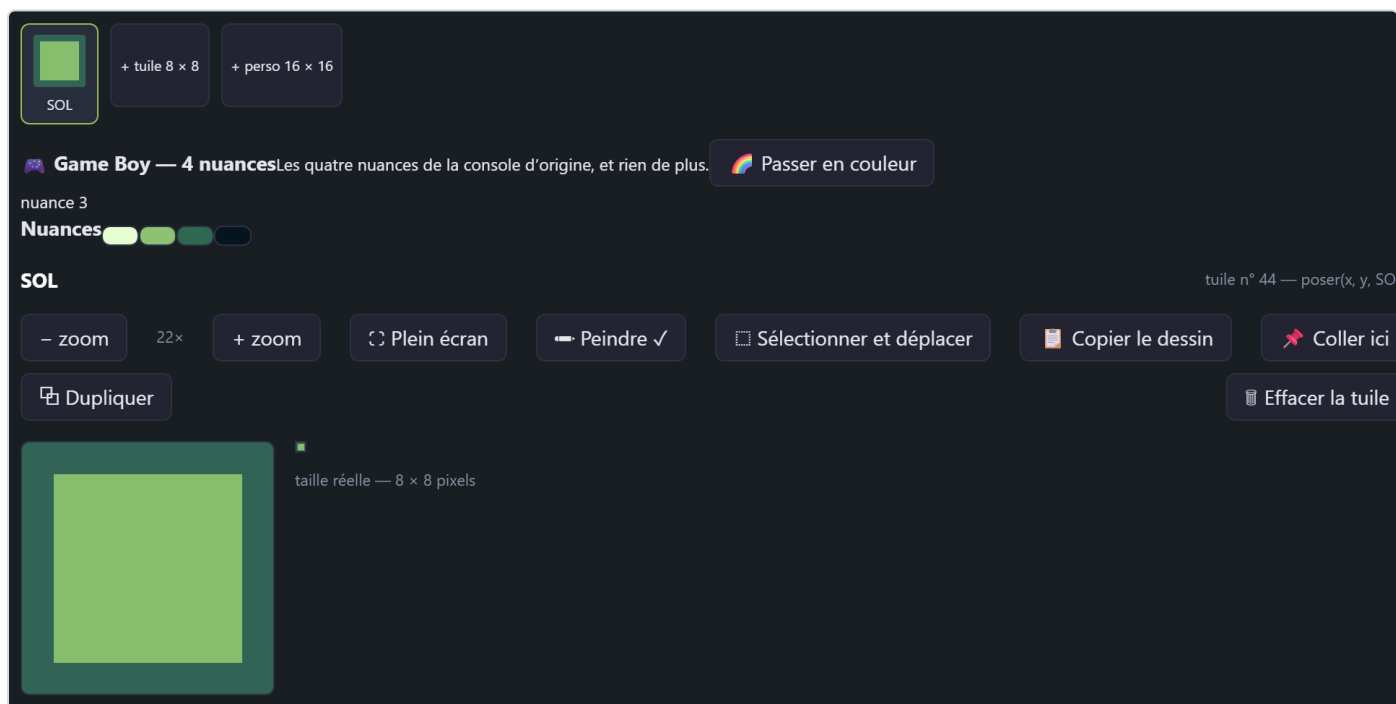
5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ le sol posé par son nom couvre ses vingt cases
- ✓ la porte écrite sur place a sa propre tuile (tuile 45)
- ✓ et le dessin redit dans l'autre écriture retrouve SOL (tuile 44)

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.



Ensuite : **19. Intégrer une carte dans son projet** — niveau 3 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++